Guia do que eu uso no Claude Code

Atualizado em 05/11/2026

Os recursos que eu tenho instalados no Claude Code, por tipo. Cada tipo abre com uma explicação curta; a ficha completa está no Kit IA.

Plugins

Pacotes que juntam skills, hooks, subagentes e conectores numa instalação só.

- **fluxo-geral** (meu): hooks que valem para qualquer projeto: formatação com ruff, proteção de segredos, confirmação antes de comandos destrutivos, notificações, passagem automática entre sessões, log de comandos e testes ao parar. Detalhes na seção Hooks.
- **superpowers** (15 skills): métodos de trabalho para programar com o Claude. Exemplos: `brainstorming` (explorar a ideia antes de construir), `test-driven-development`, `systematic-debugging`, `verification-before-completion` (provar que funciona antes de dizer que terminou), `writing-plans`.
- **mattpocock-skills** (39 skills): engenharia de verdade, sem "vibe coding". Exemplos: `grill-me` (entrevista que aperta um plano até ele ficar firme), `to-spec` e `to-tickets` (transformar conversa em especificação e tarefas), `tdd`, `code-review`, `handoff` (documento para outra sessão continuar).
- **marketing-skills** (50 skills): marketing e crescimento. Exemplos: `copywriting`, `seo-audit`, `ads`, `pricing`, `launch`, `cold-email`.
- **ui-ux-pro-max** (7 skills): design de interface com uma base de estilos, paletas, pares de fontes e gráficos. Exemplos: `ui-ux-pro-max`, `design-system`, `brand`, `slides`.
- **brag** (2 skills): transforma o projeto atual num vídeo curto de lançamento.

Hooks

Comandos que o próprio programa roda sozinho em momentos fixos (antes ou depois de uma ação, ao abrir ou fechar a sessão), sem depender de o Claude lembrar.

Meus:

- Ao abrir a sessão, atualiza o catálogo de todas as skills instaladas.
- Ao abrir a sessão, atualiza o inventário de recursos.
- Depois que a conversa é compactada, devolve ao Claude os lembretes fixos, o estado do git e os meus últimos pedidos.
- Depois de editar um arquivo, confere a sintaxe de `.sh` e `.json` e roda o eslint; se houver erro, devolve ao Claude.

Do plugin fluxo-geral:

- Ao abrir a sessão, mostra o ramo, as alterações, os commits e a última passagem automática.

- Protege segredos: bloqueia a leitura de `.env`, chaves e credenciais, e pergunta antes de editá-los. Pede confirmação antes de comandos destrutivos (`rm` recursivo, `push --force`, `reset --hard`...).
- Registra cada comando de terminal num log.
- Depois de editar um arquivo Python, roda o ruff e devolve os erros ao Claude.
- Ao terminar, roda os testes rápidos do projeto (se o projeto tiver o arquivo de testes rápidos); se falharem, o Claude continua para corrigir.
- Notificação no desktop quando o Claude pede permissão ou espera resposta, e quando uma tarefa de 2 minutos ou mais termina.
- Antes de compactar e ao fechar a sessão, grava uma passagem automática (ramo, alterações, commits, últimos pedidos).

Do plugin superpowers:

- Ao abrir a sessão, carrega a introdução do plugin, que ensina o Claude a usar as skills dele.

Subagentes

Assistentes especializados que o Claude chama para uma tarefa; cada um trabalha com o próprio contexto e devolve só o resultado.

- **investigador-de-bugs**: investiga um erro, teste falhando, resultado errado ou lentidão com método (reproduzir, hipóteses, prova) e devolve a causa com evidência e a correção mínima. Não altera o código.
- **passador-de-sessao**: reescreve o documento de retomada do projeto (o que foi feito, estado atual, próximos passos) para a próxima conversa saber onde parou.
- **pesquisador-web**: pesquisa na web em fontes primárias (documentação oficial, órgão público, fabricante) e devolve uma resposta curta com cada afirmação ligada à fonte.
- **revisor-de-codigo**: revisa as mudanças antes do commit, procurando bugs reais, mudança além do pedido, complexidade desnecessária e falta de teste. Só aponta, não altera.

Skills pessoais

Pastas com instruções que ensinam o Claude a fazer um tipo de tarefa de um jeito específico. Ele usa sozinho quando o pedido combina, ou você chama com `/nome`.

Pensamento e planejamento

- `analise-critica`: diagnóstico de uma empresa ou área com base em evidências (documentos, reuniões, dados públicos); aponta problemas com causa raiz e propõe iniciativas com 5W2H e metas SMART.
- `brainstorming`: explora a intenção, os requisitos e o desenho antes de construir qualquer coisa.
- `grill-me`: entrevista insistente para deixar um plano ou desenho mais firme.

- `wayfinder`: planeja um trabalho grande demais para uma sessão como um mapa de decisões, resolvidas uma de cada vez.
- `recursos`: depois de planejar um projeto, avalia quais hooks, subagentes, skills, plugins e conectores instalados valem a pena usar e aponta o que falta.

Desenvolvimento

- `diagnosing-bugs`: ciclo de diagnóstico para bugs difíceis e perdas de desempenho.
- `karpathy-guidelines`: regras para evitar erros comuns de IA ao programar: não complicar, mudar só o necessário, deixar claras as suposições, definir como verificar.
- `graphify`: transforma código, documentos ou imagens num grafo de conhecimento para responder perguntas sobre a estrutura de um projeto.
- `write-swift`: como escrever Swift moderno bem.

Design e interface

- `apple-design`: o jeito da Apple de desenhar interfaces e movimento, adaptado para a web.
- `emil-design-eng`: a filosofia de Emil Kowalski sobre acabamento de interface e os detalhes que fazem um software parecer bom.
- `break-ui`: tenta quebrar uma tela com dados no pior caso (nomes enormes, campos vazios, emoji, números extremos) e lista o que quebrou, com a correção.
- `prototype`: constrói várias versões diferentes de um pedaço de interface para você comparar ao vivo e escolher. Só roda quando chamada.
- `pick-ui-library`: escolhe a biblioteca certa para uma tarefa de frontend a partir de uma lista selecionada. Só roda quando chamada.
- `mobile-native`: ajustes de CSS e meta tags para um site no celular parecer um app instalado.
- `ask-sonner`: guia da biblioteca Sonner, de notificações (toasts) em React.

Animação

- `animate`: cria uma animação do zero, decidindo na ordem certa se deve animar, com que propósito, ferramenta, curva e duração.
- `animate-expo`: o mesmo para apps em React Native e Expo.
- `improve-animations`: audita as animações de um projeto e escreve um plano de melhorias priorizado. Não altera o código.
- `review-animations`: revisa código de animação com um padrão alto de qualidade.
- `find-animation-opportunities`: procura lugares que deveriam ter animação (e descarta os que não deveriam). Só propõe.
- `animation-vocabulary`: descobre o nome certo de um efeito de animação a partir de uma descrição vaga.

Vídeo (HyperFrames)

- `hyperframes`: ponto de entrada para criar, editar ou renderizar vídeos e animações com HyperFrames.

- `general-video`: vídeos mais longos ou com várias cenas: reels de marca, montagens, cartões de título.
- `hyperframes-animation`: todo o conhecimento de animação para os vídeos (regras de movimento, cenas, transições).
- `hyperframes-audio`: mixagem do áudio do vídeo (fades, volume, música sob a narração, efeitos).
- `hyperframes-cli`: o ciclo de trabalho pela linha de comando do HyperFrames (criar, conferir, prévia, renderizar, publicar).
- `hyperframes-core`: as regras de estrutura de um projeto HyperFrames.
- `hyperframes-creative`: direção criativa (paleta, tipografia, narração, ritmo).
- `hyperframes-keyframes`: zoom, movimento de câmera e outras transições quadro a quadro.
- `hyperframes-registry`: busca e instala blocos visuais prontos (efeitos, gráficos, transições).
- `hyperframes-studio`: trabalhar junto com uma pessoa no Studio do HyperFrames, do plano ao storyboard.
- `media-use`: resolve toda mídia do vídeo (música, efeitos sonoros, imagens, ícones, logos, voz).

LinkedIn

- `li-post`: escreve um post a partir de uma ideia solta, com opções de gancho, na voz da pessoa e sem cara de IA.
- `li-plan`: planeja a semana: o que postar, quando e com quem interagir.
- `li-carousel`: monta um carrossel (post em documento), slide por slide.
- `li-repurpose`: transforma um conteúdo longo (vídeo, podcast, newsletter) numa semana de posts.
- `li-comment`: escreve comentários em posts de outras pessoas, com opinião, sem parecer robô.
- `li-reply`: rascunha respostas aos comentários dos seus posts, por ordem de importância.
- `li-dm`: convites de conexão e mensagens de acompanhamento que recebem resposta.
- `li-inbox`: organiza a caixa de mensagens em contatos comerciais, recrutadores, colegas e spam, com rascunhos de resposta.
- `li-profile`: dá nota ao perfil numa escala de 100 e reescreve o que perde pontos.
- `li-audit`: analisa o que você já publicou: o que funcionou, por quê e o que parar de fazer.
- `li-human`: tira as marcas de texto de máquina de qualquer rascunho e confere se ainda parece IA.

Conectores

Ligações entre o Claude e outros serviços. Ele age com a sua conta: vê e pode alterar o que você pode ver e alterar.

- **Google Drive**: procura e lê documentos, planilhas e apresentações.
- **Gmail**: procura e lê e-mails, e prepara rascunhos.
- **Google Agenda**: consulta compromissos e sugere horários.

- **Perplexity**: pesquisa na web e responde com as fontes citadas.
- **Firecrawl**: lê páginas da web e devolve o texto limpo; faz buscas e extrai dados de vários sites.
- **Claude Docs**: documentos editáveis criados pelo Claude, que outras pessoas podem ler e comentar.

Cuidados: conecte só o que precisa; deixe em "pedir aprovação" as ações que enviam ou apagam (mandar e-mail, apagar arquivo); Firecrawl e Perplexity exigem conta própria e podem ter custo; cuidado com dados de clientes.